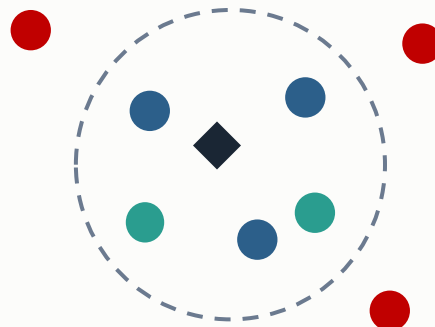


Classification with K-Nearest Neighbours (KNN)



Week 3 Laboratory · 2 hours

Your first machine-learning algorithm — built by hand.

1

**Build KNN from
scratch**

2

**Compare with
scikit-learn**

3

**Classify flower
images**

Self-Introduction

Quang Nguyen, PhD

(Van-Quang Nguyen)

Email: vanquangn@uow.edu.au

Research Interest:

- Computer Vision,
- Multimodal AI (esp. vision-language learning)
- Machine Learning

Lab Materials:

- github.com/quangvnai/csci2018



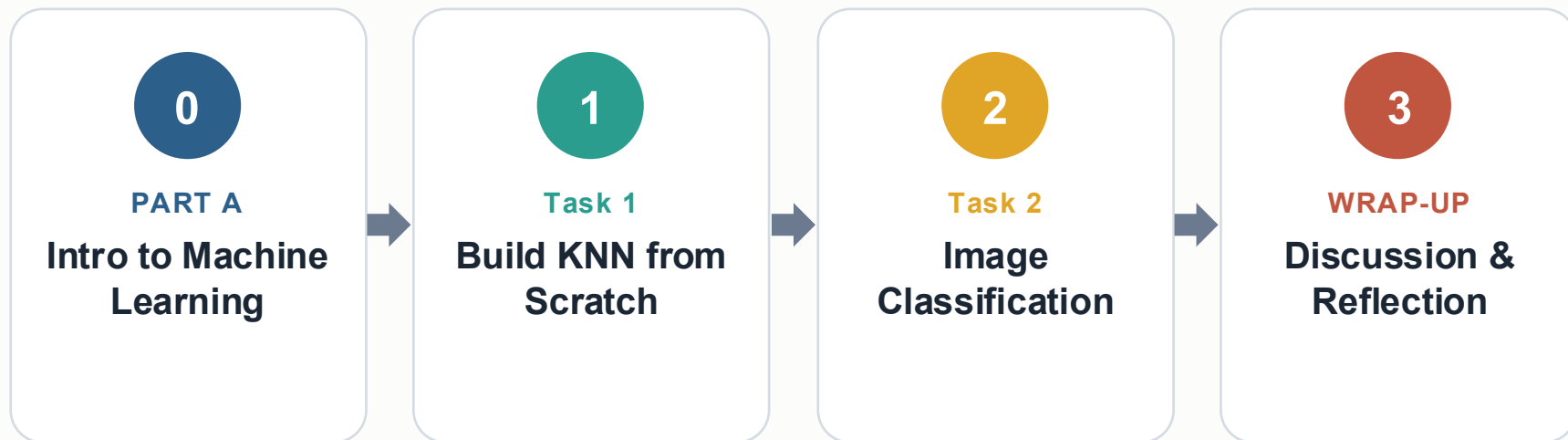
Learning Objectives

By the end of today's lab, you should be able to:

- 1 Explain **how KNN classifies** a new sample
- 2 Implement the key components of **KNN yourself**
- 3 Use KNeighborsClassifier from **scikit-learn**
- 4 Understand the role of **train, validation & test sets**
- 5 Apply KNN to **image flower classification** (Iris and Flowers data)

You'll understand KNN by building it — not just calling a library.

Today's Roadmap



PART A

0

Section 1 of 4

Introduction to Machine Learning

How machines learn from examples — and the workflow behind every ML project.

IN THIS SECTION

What is ML? · The ML workflow · Why we split data

What is Machine Learning?

TRADITIONAL PROGRAMMING

You write the rules.

Rules + Data



Output

MACHINE LEARNING

It learns the rules from examples.

Data + Labels



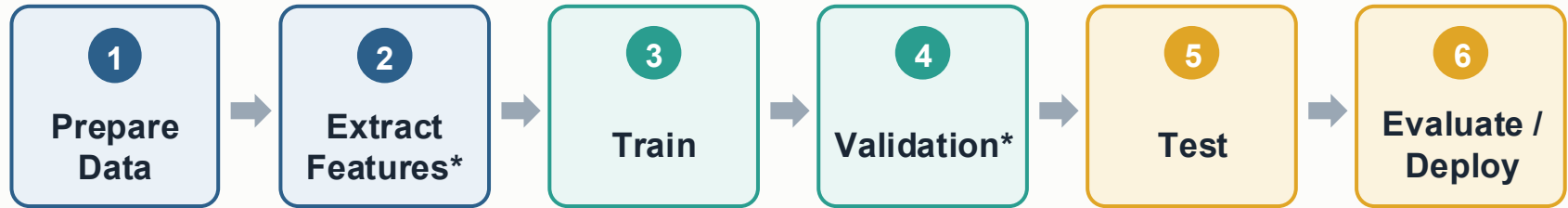
Learning Algorithm



Prediction Model

KEY IDEA ML learns patterns from examples, instead of following rules we write by hand.

The Machine-Learning Workflow



I · PREPARE DATA & FEATURES

II · TRAIN & TUNE*

III · TEST & SHIP

**optionally*

KEY IDEA Every ML project — from KNN to deep learning — follows this typical workflow.

Why Split the Dataset?

Three datasets, three different jobs — just like preparing for an exam.

DATASET	REAL-LIFE ANALOGY	PURPOSE
Training	Study materials	Learn the model
Validation	Mock exam	Choose best settings / solutions
Test	Final exam	Judge the final model — once

THINK Why shouldn't we use the final exam to decide how to study?

Task 1

1

Section 2 of 4

Task 1: Build KNN from Scratch

Meet the Iris dataset and implement K-Nearest Neighbours, step by step.

IN THIS SECTION

Iris data · Features & labels · How KNN works · Build & compare

Meet the Iris Dataset

150 labelled flower samples across
3 species: *setosa* · *versicolor* · *virginica*

One Iris flower = one sample, represented by:

4 MEASUREMENTS — FEATURES:

Sepal length

Sepal width

Petal length

Petal width



THINK Can a computer identify the species using only these 4 numbers?

Features and Labels

Each flower: four feature values in, one species label out.

FEATURES (INPUT) — what we measure



Sepal length **5.1 cm**

Sepal width **3.5 cm**

Petal length **1.4 cm**

Petal width **0.2 cm**

KNN predicts



LABEL (OUTPUT) — what we predict



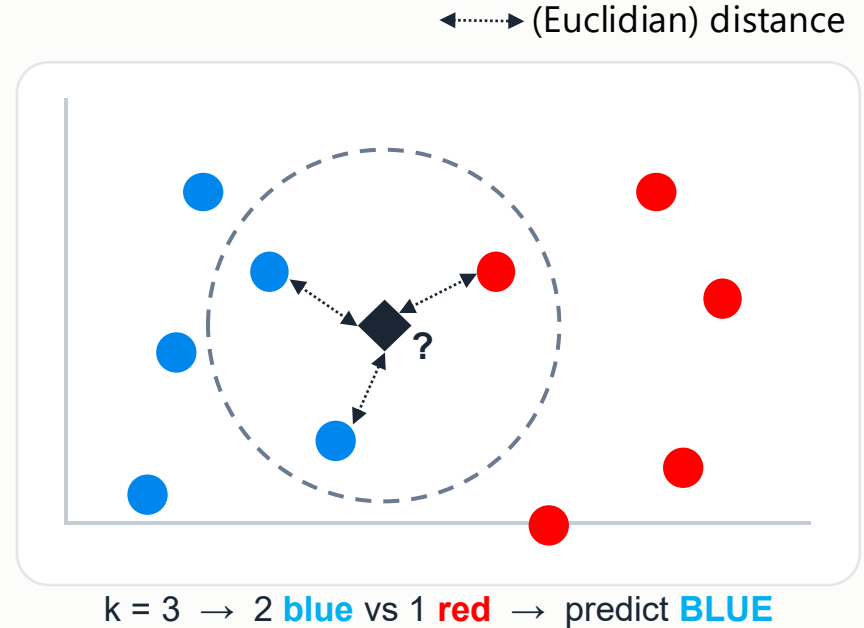
Versicolor

one of 3 possible species

KEY IDEA Machine learning uses features (X) to predict a label (y).

How Does KNN Work?

- 1 A new sample (e.g., a flower) arrives
- 2 Measure distance to every training flower
- 3 Keep the k closest neighbours
- 4 Take a majority vote
- 5 Assign the winning species

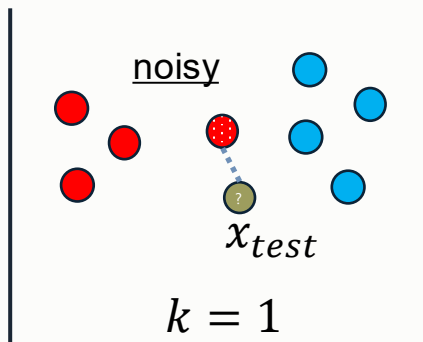


KEY IDEA KNN predicts a label from the most similar training samples — no training, just distances.

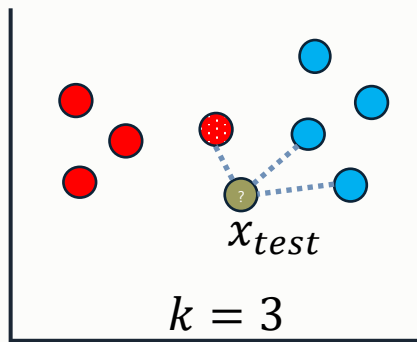
Which Value of K is Optimal?

- **K = 1:** Sensitive to noise and outliers.
- **Even K:** May lead to ties during majority voting.
- **Large K:** May oversimplify the decision boundary.

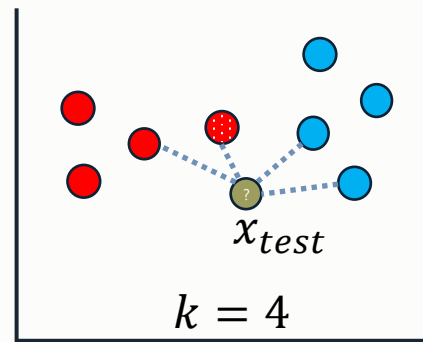
→ **Optimal K** is a **hyperparameter**, determined experimentally using the *validation set*.



Nearest neighbor is {red};
So, x_{test} is predicted as red.



Nearest neighbors are {red, blue, blue};
So, x_{test} is predicted as red.



Nearest neighbors is {red, red, blue, blue};
The prediction is undefined!

Build KNN from Scratch

No libraries yet — you'll write each piece of the algorithm yourself.

1

Euclidean distance

`euclidean_distance(a, b)`

How far apart are two flowers?

2

Find nearest neighbors

`Find_neighbors(X_train, x, k)`

Keep the k closest samples.

3

Majority vote

`vote(neighbors, y_train)`

Count labels — pick the winner.

4

MyKNNClassifier

`MyKNNClassifier(k=3)`

Wrap it all into one function.

OPEN THE NOTEBOOK → build KNN one function at a time · ~45 min

Compare with scikit-learn

Same algorithm — 3 lines instead of 50. Does your version agree?

● ● ● knn_from_scratch.py

```
# your implementation of MyKNNClassifier
# ...
knn_scratch = MyKNNClassifier(k=5)
knn_scratch.fit(X_train, y_train)
pred = knn_scratch.predict(X_test)
```

~50 lines you wrote by hand

● ● ● knn_sklearn.py

```
from sklearn.neighbors import KNeighborsClassifier
knn_sklearn = KNeighborsClassifier(k=5)
knn_sklearn.fit(X_train, y_train)
pred = knn_sklearn.predict(X_test)
```

3 lines — same result ✓

KEY IDEA Libraries save time — but now you know exactly what happens inside.

TASK 2

2

Section 3 of 4

Task 2: KNN for Image Classification

Turn images into features and apply KNN to a real image problem.

IN THIS SECTION

Images as numbers · Feature extraction · Tuning K · Evaluation

Meet the **Flowers** Dataset

Real-world flower photos gathered from the web — the classification task for today's MLP and CNN.

4,317

images

5

flower classes

800–1,000+

photos per class

3

web sources

Daisy

Tulip

Rose

Sunflower

Dandelion

Why? Automating flower recognition speeds up search for the public and floriculture — think digital libraries. Apps like Bing and Google Lens already use it.



various resolutions & proportions

Sources: Flickr · Google Images · Yandex Images

From Numbers to Images

Both are numbers — but KNN needs a short, fixed-length vector.

TABULAR DATA (Task 1)

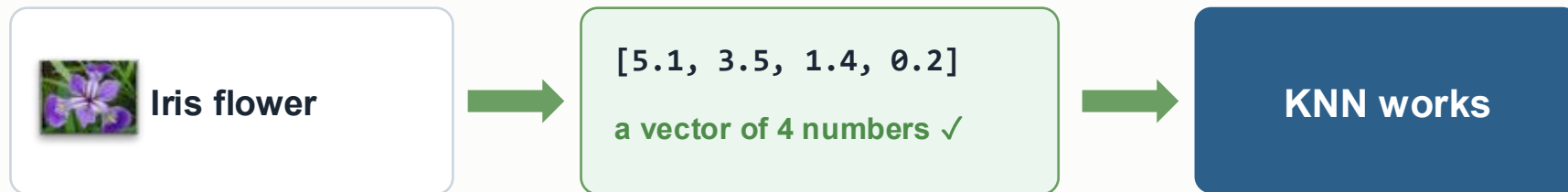
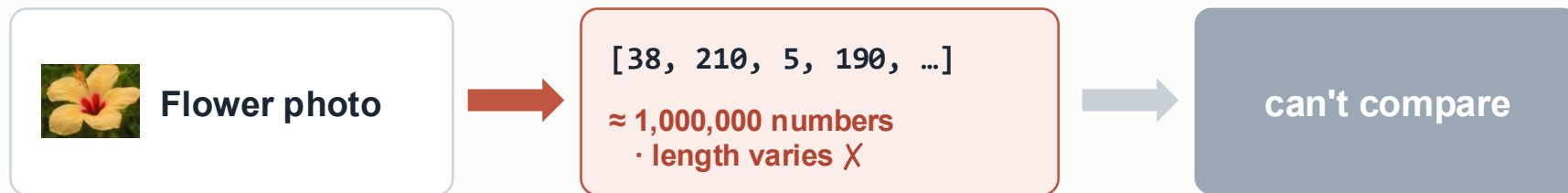


IMAGE DATA (Task 2)

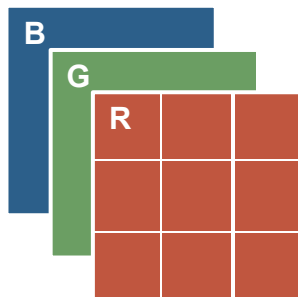


THINK An image is already numbers — so why not feed the raw pixels straight to KNN?

How a Computer Sees an Image

An image is just a grid of pixel values — three per pixel.

HOW AN IMAGE IS STORED



RGB image = 3 colour channels

height × width × 3 numbers

One pixel = 3 numbers (R, G, B)

So the whole image = height × width × 3.

$$600 \times 600 \times 3$$

1,080,000
numbers

$$600 \times 800 \times 3$$

1,440,000
numbers

Different sizes → different-length vectors
— KNN can't compare them.

KEY IDEA Raw pixels make a **giant** vector whose *length changes* with the image size.

Feature Extraction: Colour Histograms



WHY A COLOUR HISTOGRAM?



Fixed size

always the same number of bins



Size-invariant

600×600 or 600×800 → same length



Colour-aware

captures which colours dominate,
robust to pixel shift, flipping, rotations, etc.



dark ▶ light

dark ▶ light

Different colours → clearly different histograms.

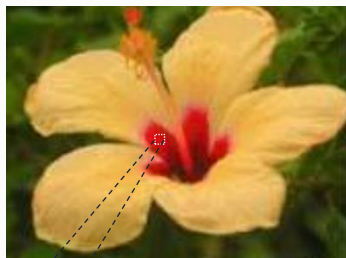
KEY IDEA A colour histogram is a fixed-size, size-independent summary of an image's colours.

3-D colour histogram: How to Describe the Colours in an Image?

One simple way: build a 3-D colour histogram.

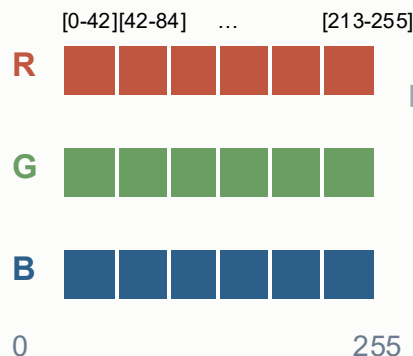
① AN IMAGE

150 × 150 px · 3 channels (R,G,B)



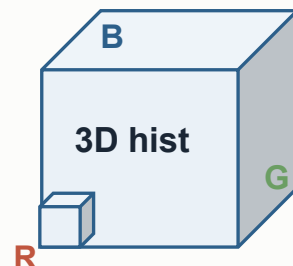
one pixel = (R, G, B)
(220, 60, 30)
each value 0–255

② SPLIT EACH CHANNEL INTO 6 BINS



6 bins per channel

③ COMBINE INTO A 3-D HISTOGRAM & COUNT



$6 \times 6 \times 6 =$
216 color bins

Each bin stores the count.

Pixel (220, 60, 30)

R→bin 5 G→bin 1 B→bin 0

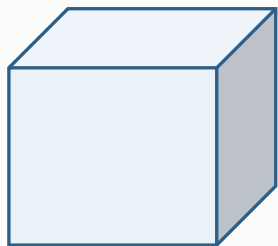
`hist[5][1][0] += 1`

KEY IDEA A 3-D colour histogram counts each Red×Green×Blue combination — that's $6 \times 6 \times 6 = 216$ color bins, not $6+6+6$ (channel-independently).

3D Histogram: How Can KNN Compare Two Images?

Turn each histogram into one fixed-length feature vector.

START · ONE IMAGE



3D histogram

$6 \times 6 \times 6 = 216$ counts

① NORMALIZE TO 0-1

before

5

18

42

100

→ ÷ by the largest value

after

0.05

0.18

0.42

1.00

Different images have different pixel counts — scaling makes them comparable.

✓ Robust to **scaling**

② FLATTEN INTO A VECTOR

0.00

0.15

0.32

0.08

⋮

0.91

216 numbers

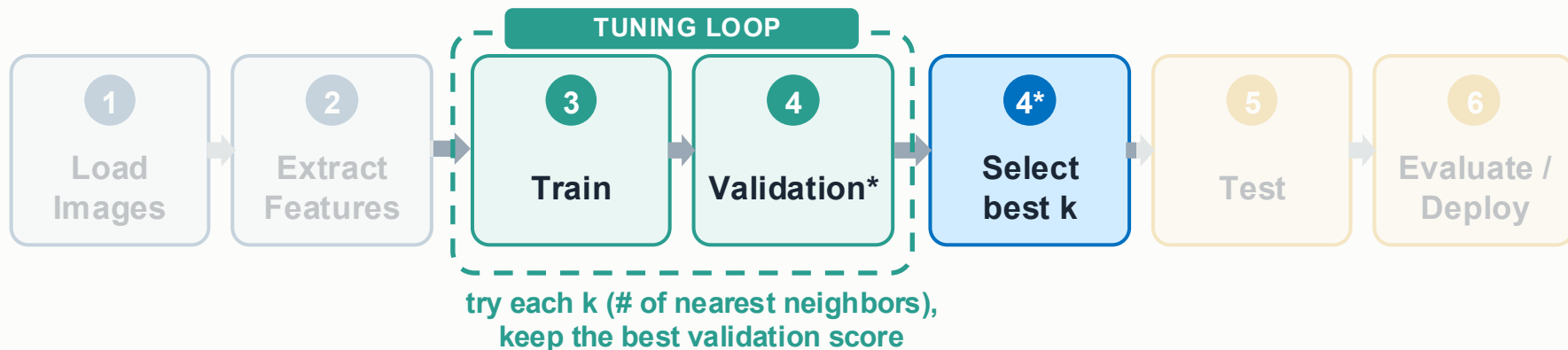
KNN

Euclidean distance

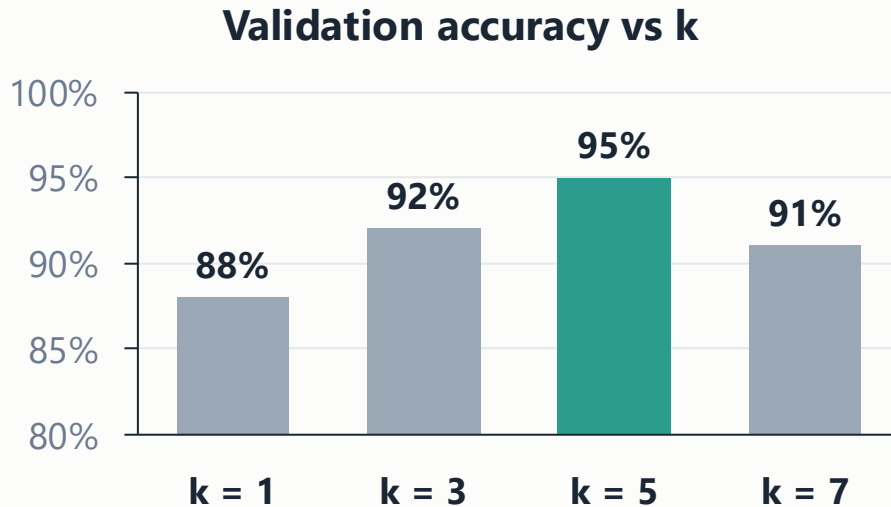
KEY IDEA Normalize + Flatten turns each image into one 216-D vector — KNN then compares images by Euclidean distance.

A More Realistic ML Workflow

The same pipeline as before — now with *a real loop to tune k*.



Why Do We Need a Validation Set?



HOW WE CHOOSE **k**

- 1 Try $k = 1, 3, 5, 7$ on the **validation set**
- 2 **$k = 5$ scores best — 95%**
- 3 Lock in $k = 5$, **evaluate on the test set**

Never tune on the test set — that's cheating.

KEY IDEA Validation lets us choose k without ever peeking at the test set.

Evaluating the Model

Accuracy alone hides what kind of mistakes a model makes.

Accuracy

overall % of correct predictions

Precision

of those predicted X, how many were right

Recall

of the actual X, how many we caught

F1-score

balance of precision & recall

Confusion Matrix

		PREDICTED →		
		A	B	C
GROUND TRUTH (ACTUAL) ↓	A	15	0	0
	B	0	14	1
	C	0	2	13

Diagonal = correct. Off-diagonal = mistakes.

KEY IDEA Good evaluation reveals not just how often, but how the model is wrong.

WRAP-UP



Section 4 of 4

Review & Reflection

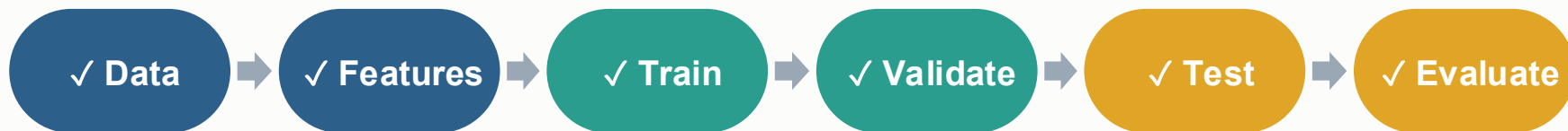
Step back and see KNN as one algorithm inside a workflow you'll reuse.

IN THIS SECTION

What we learned · **Reflection questions**

What Have We Learned?

One lab, the full pipeline — every stage now checked off.



You ran the entire machine-learning workflow — end to end.

...using **K-Nearest Neighbours** as your first algorithm.

Reflection Questions

Before you leave, make sure you can answer these:

- 1 How does KNN make a prediction?
- 2 Why do we split data into train, validation, and test sets?
- 3 Why can't KNN classify raw images directly?
- 4 What happens if k is too small or too large?
- 5 How did your from-scratch KNN compare with scikit-learn?